ENE0167 2026.2 Controle Digital – Prof. Adolfo Bauchspiess

Controle PI Anti-Windup Digital de Processo Térmico com ESP32 – Grupo 2

Pedro [Sobrenome] * **[Nome do colega]** **

* *[Matrícula], [Curso], UnB, pviana2000@gmail.com*
** *[Matrícula], [Curso], UnB, [e-mail]*

Abstract: A digital anti-windup PI controller was identified, designed and implemented on an ESP32 for a thermal process (LM35 heated by PWM-driven resistors, with a fan as disturbance). The plant was identified from a 9-hour PRBS experiment and the controller was designed by root locus with $T_s = T_d/2$. Experiments with and without anti-windup show overshoot of about 0 % versus 13–22 % in saturated 10 °C reference steps.

Resumo: Este relatório descreve a montagem, a identificação, o projeto e a implementação de um controlador PI digital com anti-windup para um processo térmico formado por um sensor LM35 aquecido por três resistores acionados por PWM de 16 bits em uma ESP32, com um cooler como perturbação. O modelo P2DZ foi identificado (`procest`) a partir de uma coleta PRBS de 9,07 h em torno do ponto de operação de 50 °C, com fit de 86 % na validação. O PI foi projetado no lugar das raízes discreto com $T_s = T_d/2 = 2{,}629$ s para $M_p = 10\,\%$ e $t_p = 25$ s. A saturação do atuador foi escolhida experimentalmente ($U_{max} = 21000$ contagens) para que a referência de 60 °C fosse alcançável mesmo com a perturbação, mas os degraus ainda saturassem. No ensaio da Figura 5 do roteiro, o anti-windup reduziu o sobrepasso de 13–22 % para menos de 1 %, com ISE de 2998 °C^2·s (faixa típica: 2000–4000). A perturbação constante foi rejeitada sem erro em regime; a triangular deixou erro residual de até ±0,8 °C, limite esperado de um controlador com um único integrador.

Keywords: Digital control; PI controller; anti-windup; system identification; thermal process; ESP32.

Palavras-chaves: controle digital; PI anti-windup; identificação PRBS; lugar das raízes; processo térmico; ESP32.

1. INTRODUÇÃO

O objetivo do Prj1CDig (Bauchspiess, 2026) é identificar um processo térmico em malha aberta e controlá-lo com um PI digital anti-windup projetado pelo lugar geométrico das raízes (LGR), verificando o projeto em simulação e experimentalmente. As especificações do grupo 2 estão na Tabela 1.

Tabela 1. Especificações do grupo 2 e requisitos do roteiro.

Ponto de operação (P.O.)	50 °C
Sobrepasso M_p	10 %
Tempo de pico t_p	25 s
Amostragem da identificação T_i	1 s
Amostragem do controle T_s	$T_d/2$
Resolução da temperatura	$\leq 0{,}1$ °C
Perturbações	constantes e triangulares
Frequência do cooler ω_0	0,065 rad/s

2. MONTAGEM E INSTRUMENTAÇÃO

2.1 Componentes e ligações

A bancada segue o item 1 do procedimento do roteiro (Tabela 2 e Figura 1). Os três resistores de 150 Ω ficam em paralelo, encostados ao encapsulamento do LM35 do processo para melhorar a transferência de calor. O LM35 do ambiente fica a cerca de 15 cm do aquecedor, fora do fluxo de ar do cooler, como pede o roteiro (“a temperatura ambiente deve ser medida longe da perturbação”). A montagem fica exposta ao ar da sala, sem caixa, para que o modelo inclua a influência do ambiente.

2.2 Medição de temperatura e filtros

Damper RC. Cada LM35 tem na saída um resistor de 75 Ω em série com um capacitor de 1 µF para o terra. É o circuito “R-C damper” do datasheet (Texas Instruments, 2017), recomendado quando a saída do sensor alimenta cabos ou cargas capacitivas e há fontes de ruído próximas (o motor do cooler e o PWM do aquecedor). Sua frequência



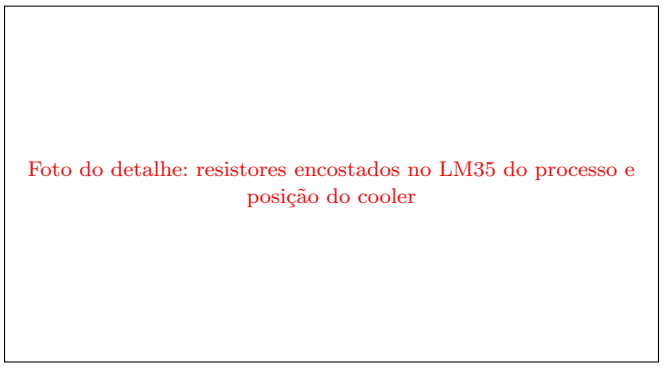


Figura 1. Bancada do processo térmico. [inserir fotos]

de canto, $1/(2\pi \cdot 75 \cdot 10^{-6}) \approx 2{,}1\,\text{kHz}$, está muito acima da dinâmica térmica: ele estabiliza o sensor, mas não é o filtro do sinal.

Filtragem digital. A filtragem é feita no firmware: cada amostra é a média de 400 ms de conversões do ADC1 (cerca de 1600 leituras por sensor). A janela de 400 ms contém um número inteiro de ciclos de 50 Hz (20) e de 60 Hz (24), que se cancelam na média, e a fase de cada rajada de leituras é sorteada dentro do período de 1 ms do PWM do aquecedor, evitando que a média fique "presa" sempre no mesmo ponto da onda.

ADC. Usa-se o ADC1 (o ADC2 conflita com o rádio) com atenuação de 2,5 dB, cuja faixa útil (até ≈1250 mV, ou 125 °C) cobre com folga o limite de segurança de 95 °C; com 0 dB a faixa terminaria perto desse limite. A curva de calibração de fábrica gravada em eFuse é uma reta, $V\,[\text{mV}] = 0{,}31033\,\text{raw} + 77{,}83$, aplicada à média em ponto flutuante para não perder a resolução ganha na média (Espressif Systems, 2026). Um LSB vale 0,31 mV, ou 0,031 °C.

Tabela 2. Componentes e ligações da bancada.

Item	Especificação / ligação
Microcontrolador	ESP32-WROOM-32 DevKit, ESP-IDF v6.1
Sensor do processo T_p	LM35, saída no GPIO35 (ADC1_CH7)
Sensor ambiente T_a	LM35, saída no GPIO34 (ADC1_CH6)
Alimentação dos LM35	5 V da USB (4,66 V medidos)
Aquecedor	$3 \times 150\,\Omega$, 1/4 W, em paralelo ($50\,\Omega$), 12 V
PWM do aquecedor	GPIO25, LEDC 16 bits, 1 kHz
Cooler (perturbação)	12 V, PWM no GPIO26, 10 bits, 25 kHz
Chaveamento	[modelo dos transistores]
Damper RC nos LM35	$75\,\Omega$ ($2 \times 150\,\Omega$ em paralelo) + $1\,\mu$F
Fonte	12 V DC

Resultado. Com o aquecedor desligado, o desvio-padrão de T_p nas últimas 100 amostras foi de 0,047 °C (0,036 °C com o cooler ligado), já incluindo a deriva lenta do ambiente. Atende ao requisito de resolução de 0,1 °C sem conversor externo.

2.3 Atuadores

Aquecedor, PWM de 16 bits. O roteiro pede PWM de 16 bits (item 2) para "evitar propagação de erro". No LEDC da ESP32, 16 bits exigem $f \le 80\,\text{MHz}/2^{16} = 1{,}22\,\text{kHz}$, por isso se usa 1 kHz, muito acima da banda térmica. Com o ganho identificado (Seção 3), uma contagem de 16 bits muda T_p em ≈0,002 °C. Com 10 bits seria ≈0,1 °C por passo, da ordem da resolução exigida, o que levaria o PI a um ciclo-limite entre dois valores vizinhos. O sinal u é tratado em contagens (0–65535) em todo o projeto, como na tabela de variáveis do roteiro.

Cooler. PWM de 10 bits a 25 kHz, a frequência usual de ventoinhas, fora da faixa audível. Medida a zona morta: o motor parte em $w \approx 530$ (52 %). As perturbações usam $w \in [560; 680]$: constante em 680 e triangular $w = 620 + 60 \cdot \frac{2}{\pi}\arcsin(\sin \omega_0 t)$, com período $2\pi/\omega_0 = 96{,}7\,\text{s}$. Em malha aberta a 50 °C, $w = 680$ baixou T_p de 49,7 para 45,3 °C em 6 min, tendendo a ≈5 °C de queda.

2.4 Saturação do atuador

Cada resistor dissipa $P_R = d \cdot 12^2/150 = 0{,}96\,d\,\text{W}$, sendo d o ciclo de trabalho. O limite U_{max} (o "Sat" do roteiro, usado igualmente no firmware e no MATLAB) foi escolhido em três etapas (Tabela 3). A conclusão é que ele precisa ficar numa janela:

$$u_{\text{regime,máx}} < U_{max} < u_{\text{pico}}, \tag{1}$$

onde $u_{\text{regime,máx}}$ é o u que mantém 60 °C com o cooler em 680 e $u_{\text{pico}} \approx u_0 + K \cdot 10\,^\circ\text{C}$ é o que o PI pede no instante de um degrau de 10 °C. Abaixo da janela a referência não é alcançada; acima dela o atuador nunca satura e o efeito do anti-windup não aparece.

Tabela 3. Escolha experimental de U_{max}.

U_{max}	d / P_R	Resultado
17000	26 % / 0,25 W	potência nominal. No ensaio da Fig. 5, u ficou preso em 17000 nos patamares de 60 °C com cooler e T_p parou em 55,5–56,2 °C (ISE = 9372).
34000	52 % / 0,50 W	simulação (seção 7 do script): $u_{\text{pico}} \approx 30600 < U_{max}$, sem saturação; respostas com e sem anti-windup idênticas.
21000	**32 % / 0,31 W**	teste no pior caso (60 °C, $w = 680$, Fig. 2): satura 26 s e depois se mantém em $u \approx 19560$ (folga de 7 %), com $T_p = 59{,}9 \pm 0{,}1$ °C. **Adotado.**

O firmware também desliga o aquecedor se $T_p > 95$ °C.

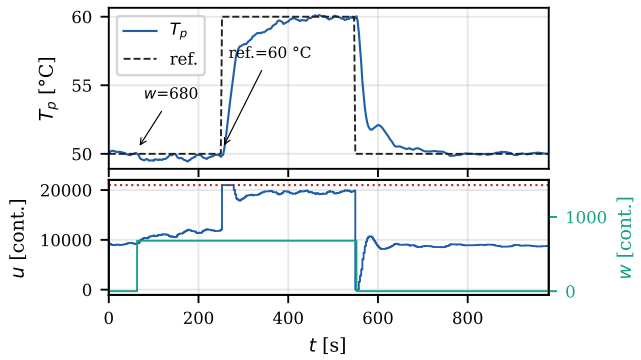


Figura 2. Teste do pior caso com $U_{max} = 21000$: cooler em 680 a partir de 63 s e degrau de referência para 60 °C em 252 s. O atuador satura só no transitório.

2.5 Firmware e aquisição

O firmware (ESP-IDF, Apêndice A) tem cinco modos selecionados por comandos na serial: sensores, malha aberta, PRBS, malha fechada e o perfil de ensaio da Fig. 5 do roteiro, executado pela própria ESP com o cálculo do ISE. A amostragem é de 1 s na identificação e T_s no controle. Um script em Python grava cada amostra em CSV e mostra T_p, T_a, u e w em tempo real. A coleta de 9 h foi registrada sem nenhuma amostra perdida ou inválida.

3. IDENTIFICAÇÃO DO PROCESSO

3.1 Ponto de operação e excursão

Em malha aberta, $u = 11000$ levou T_p a 52,1 °C, $u = 10000$ a 49,6 °C e $u = 10200$ a 49,8 °C ($T_a \approx 27{,}8$ °C), com 15–20 min de espera em cada valor. Adotou-se $u_0 = 10200$ (15,6 % de duty). Desses pontos, o ganho local é ≈0,0025 °C/contagem, e a excursão $\Delta u = 5/0{,}0025 = 2000$ produz os ±5 °C pedidos no item 2 do roteiro.

3.2 Coleta PRBS

O PRBS é gerado na ESP por um registrador de deslocamento de 9 bits com polinômio $x^9 + x^5 + 1$, que produz a sequência de comprimento máximo de 511 bits. Cada bit dura 60 s, o que gera patamares de 60 a 540 s e excita tanto o polo rápido quanto o lento. O aquecedor alterna entre $u_0 \pm \Delta u$ (8200 e 12200). Antes da sequência, u fica 2000 s em u_0, trecho descartado na identificação (`transit` = 2000, como no script do professor). A coleta durou $2000 + 511 \cdot 60 = 32660$ s (9,07 h), com $T_i = 1$ s (Fig. 3).

T_p variou entre 45,9 e 56,1 °C, com média de 51,3 °C. A média ficou 1,3 °C acima do P.O. porque a convecção natural da madrugada diferiu da noite em que u_0 foi medido: com u médio constante, a média horária de T_p variou entre 49,8 e 52,5 °C. T_a ficou entre 26,9 e 28,3 °C, com uma queda de ≈0,9 °C ao amanhecer.

3.3 Estimação do modelo P2DZ

O modelo do roteiro é

$$G(s) = \frac{T_p(s)}{U(s)} = K_p \frac{1 + T_z s}{(1 + T_{p1} s)(1 + T_{p2} s)} e^{-T_d s}. \qquad (2)$$

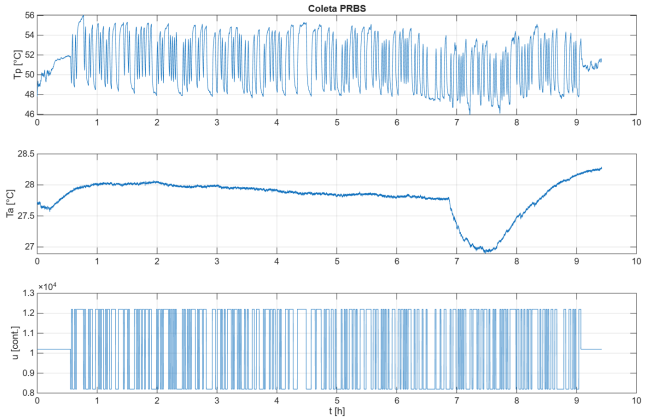


Figura 3. Coleta PRBS: T_p, T_a e u (9,07 h, $u_0 = 10200$, $\Delta u = 2000$).

Ele foi estimado com o `procest` (The MathWorks, 2026; Ljung, 1999), com três decisões:

Saída $T_p - T_a$. O roteiro modela o efeito do ambiente por $G_a(s) = T_p(s)/T_a(s)$ com ganho DC unitário. Subtrair T_a remove da identificação as variações lentas do ambiente, como a queda ao amanhecer, que u não explica.

Chute inicial e limites. Sem valores iniciais, o `procest` converge para um mínimo local sem significado físico ($T_z = 0$, $T_{p1} \approx 38$ s, $T_{p2} \approx 2{,}5$ s, $T_d \approx 2{,}2$ s) e fit menor. Iniciou-se em $K_p = 0{,}002$ (ganho da etapa anterior), $T_{p1} = 300$ s, $T_{p2} = 40$ s, $T_z = 200$ s e $T_d = 4$ s (faixas do roteiro), com limites largos (por exemplo, $50 \le T_{p1} \le 2000$ s). Partidas diferentes levaram ao mesmo mínimo.

Validação cruzada. Foram testadas 19 partições identificação/validação e escolhido o modelo de maior fit em `compare`.

Tabela 4. Modelo identificado ($y = T_p - T_a$, u em contagens).

Parâmetro	Valor	Roteiro
K_p	0,00320 °C/cont.	–
T_z	821,9 s	T_{p2}–T_{p1}
T_{p1}	1553,9 s	200–400 s
T_{p2}	34,46 s	20–50 s
T_d	5,257 s	3–8 s
Fit (75 % validação)	86,4 %	–

O melhor modelo (Tabela 4, Fig. 4) foi identificado com só 25 % dos dados (2,1 h) e validado nos 75 % restantes. O fit de todas as partições ficou entre 70 e 86 %. O modelo reproduz toda a dinâmica rápida; o que fica sem explicação são derivas lentas da convecção do ar, que nenhuma entrada medida contém.

T_{p1} ficou acima da faixa do roteiro, e o par lento (T_z, T_{p1}) é pouco determinado: outras partições e pontos de partida dão T_{p1} entre ≈1500 e 3400 s com $T_z/T_{p1} \approx 0{,}5$ e praticamente o mesmo fit. Sem isolamento, esse modo lento absorve o aquecimento do entorno. Já a parte rápida, que define o controle, é consistente em todas as variantes: $K_p T_z/T_{p1} \approx 0{,}0017$ °C/cont., $T_{p2} \approx 34$–35 s e $T_d \approx 5{,}2$–5,3 s. Em escala de 10 bits (×64), o ganho rápido equivale a 0,11 °C/contagem, coerente com a faixa $K_p = 0{,}05$–0,1

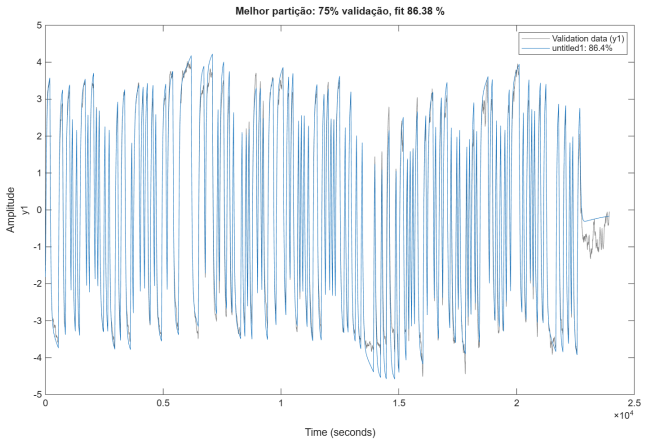


Figura 4. Validação: identificação com 25 % dos dados, validação nos 75 % restantes (y sem média). Fit de 86,4 %.

do roteiro. O PI final, aplicado a três variantes do modelo, dá M_p entre 8,6 e 10 % e $t_p = 23{,}7\,\text{s}$ em todas.

4. PROJETO DO CONTROLADOR

4.1 Polo dominante

De $M_p = 10\,\%$ e $t_p = 25\,\text{s}$: $\zeta = -\ln M_p/\sqrt{\pi^2 + \ln^2 M_p} = 0{,}5912$, $\omega_n = \pi/(t_p\sqrt{1-\zeta^2}) = 0{,}1558\,\text{rad/s}$, e $s_d = -0{,}0921 \pm j0{,}1257$.

4.2 PI contínuo (itens 4 e 5 do roteiro)

Para $C(s) = K_P(s+a)/s$, a condição de ângulo em s_d posiciona o zero em $a = 0{,}0679\,\text{s}^{-1}$ e a de módulo dá $K_P = 2150\,\text{cont./°C}$ ($K_I = K_P a = 145{,}9\,\text{cont./(°C·s)}$). A simulação linear desse ponto de partida dá $M_p = 38\,\%$ e $t_p = 22{,}7\,\text{s}$: o zero do PI e os polos adicionais aumentam o sobrepasso, e o ajuste final é feito no sisotool. [inserir figura do sisotool contínuo, valores finais de K_P e K_I e a verificação no Simulink (`Prj1CDig.slx`) com saturação e anti-windup]

4.3 PI discreto (itens 6 e 7)

Período de amostragem. $T_s = T_d/2 = 2{,}629\,\text{s}$ (item 6a). O atraso foi representado como exatamente duas amostras: com T_d/T_s fracionário (1,997), a discretização ZOH introduz um zero espúrio muito afastado (em torno de $z \approx -300$), que não altera a resposta mas distorce o LGR.

Projeto. Com $G(z)$ obtida por ZOH, o polo desejado é $z_d = e^{s_d T_s} = 0{,}7425 \pm j0{,}2547$. Para

$$C(z) = K\frac{z - z_0}{z - 1}, \tag{3}$$

a condição de ângulo dá $z_0 = 0{,}8849$ e $K = 1935$, com $M_p = 27{,}2\,\%$ e $t_p = 26{,}3\,\text{s}$. Refinou-se então (K, z_0) por busca em grade em torno desse ponto, minimizando os desvios de M_p e t_p na resposta ao degrau:

$$K = 2039{,}96\ \text{cont./°C}, \qquad z_0 = 0{,}94216, \tag{4}$$

com $M_p = 10{,}1\,\%$ e $t_p = 23{,}7\,\text{s}$ na simulação linear. Os polos de malha fechada (Fig. 5) são $0{,}7033 \pm j0{,}3185$ ($\zeta = 0{,}52$, $\omega_n = 0{,}189\,\text{rad/s}$), 0,946 (quase cancelado pelo zero do PI em 0,942), 0,9968 (quase cancelado pelo zero da planta) e $-0{,}42$. Os pares polo-zero próximos alteram a resposta: os polos dominantes ficam um pouco abaixo de $\zeta = 0{,}59$ justamente para que a resposta completa tenha $M_p = 10\,\%$. [inserir figura do sisotool discreto]

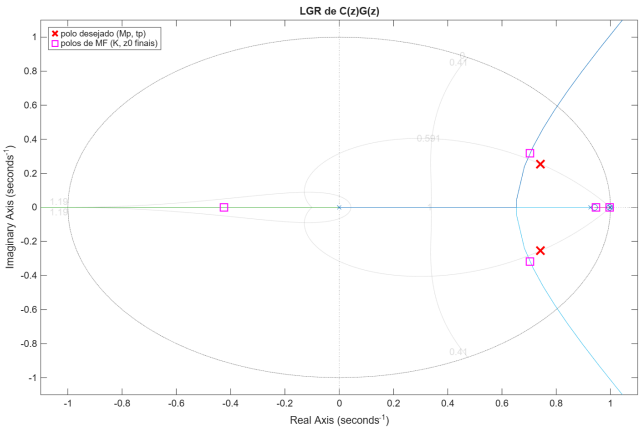


Figura 5. LGR de $C(z)G(z)$ com a grade de ζ e ω_n: polo desejado ($\times$) e polos de malha fechada com o PI final ($\square$).

Lei de controle e anti-windup. O PI é implementado como

$$u(k) = K\,e(k) + I(k), \qquad u_a(k) = \text{sat}_{[0,\,U_{max}]}\big(u(k)\big), \tag{5}$$

$$I(k+1) = I(k) + \delta(k)\,K(1 - z_0)\,e(k), \tag{6}$$

que é (3) na forma $K + K(1-z_0)/(z-1)$, com $\delta(k) = 0$ se o anti-windup está ligado e ($u > U_{max}$ com $e > 0$) ou ($u < 0$ com $e < 0$), e $\delta(k) = 1$ caso contrário. O anti-windup por integração condicional (Åström and Hägglund, 2006) congela o integrador quando o atuador está saturado e o erro empurra para mais saturação. Ao entrar em malha fechada, I é iniciado com o u atual (partida sem solavanco).

4.4 Simulação com saturação

A seção 7 do script simula $G(z)$ em torno do ponto de operação medido ($u_0 = 10200$), com a mesma lei do firmware (Tabela 5, Fig. 6). No degrau de 10 °C, o da Fig. 5 do roteiro, o integrador sem anti-windup acumula durante a saturação e o sobrepasso sobe de 0,5 para 13,5 %. No de 5 °C a saturação dura só 3 s e a diferença é pequena. [inserir a verificação no Simulink (item 7) com o PI discreto]

Tabela 5. Simulação com saturação em $U_{max} = 21000$.

	Com AW		Sem AW	
Degrau	M_p	satur.	M_p	satur.
5 °C	4,8 %	3 s	9,7 %	3 s
10 °C	0,5 %	16 s	13,5 %	29 s

5. RESULTADOS EXPERIMENTAIS

5.1 Ensaio do roteiro (Figura 5)

O perfil do item 8 foi executado pela ESP duas vezes, com e sem anti-windup (Fig. 7): a referência alterna

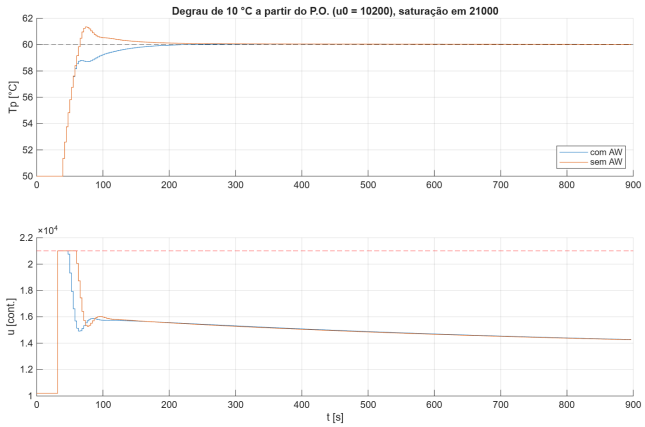


Figura 6. Simulação do degrau de 10 °C (50→60 °C em $t = 30$ s) com saturação em 21000, com e sem anti-windup (seção 7 do script).

entre 50 e 60 °C a cada 400 s; o cooler fica desligado até 1000 s, constante em 680 até 1800 s, triangular até 2600 s e constante até 3600 s. O ISE é calculado entre 2100 e 3000 s.

Tabela 6. Degraus de subida (50→60 °C): sobrepasso nos primeiros 150 s, tempo saturado e tempo até 90 % do degrau.

		Com AW			Sem AW		
#	Cooler	M_p	sat.	t_{90}	M_p	sat.	t_{90}
1	off	0,0 %	13 s	31 s	14,3 %	21 s	29 s
3	680	0,0 %	24 s	88 s	13,5 %	55 s	46 s
5	tri.	0,9 %	26 s	74 s	21,6 %	58 s	45 s
7	680	0,0 %	24 s	84 s	14,3 %	81 s	50 s

Anti-windup. Em todos os degraus de subida o atuador saturou em 21000 (Tabela 6, Fig. 9). Sem anti-windup, o integrador acumulou erro durante a saturação: o aquecedor ficou saturado de 1,6 a 3,4 vezes mais tempo e o sobrepasso foi de 13,5 a 21,6 %, contra no máximo 0,9 % com anti-windup. Nas descidas, que saturam em $u = 0$, o afundamento abaixo de 50 °C foi de 5 a 13 % sem anti-windup e de no máximo 4,8 % com ele. Com anti-windup a chegada final é mais lenta (90 % em 74–88 s com o cooler, contra 45–50 s sem), porque o integrador congelado precisa voltar a acumular depois que o atuador sai da saturação.

Simulação × experimento. No degrau 1 (sem cooler, como na simulação), o sobrepasso sem anti-windup foi de 14,3 % (simulado: 13,5 %) e o tempo saturado, de 13 s com e 21 s sem anti-windup (simulado: 16 e 29 s).

ISE. 2997,7 °C²·s com anti-windup e 2937,6 sem, ambos na faixa típica do roteiro (2000–4000). O ISE quase não distingue as duas versões: na janela de 2100 a 3000 s, a subida mais rápida sem anti-windup compensa o sobrepasso. M_p e o tempo saturado mostram melhor o benefício.

Verificação no MATLAB. A seção 9 do script (Tabela 7, Fig. 8) confirma os valores do firmware. Sem anti-windup, o pico vem 39 s após o 1º degrau (sobrepasso de fato, 14,3 %) e o aquecedor satura 2,3 vezes mais tempo. Com anti-windup, a seção 9 busca o pico em todo o patamar de 400 s e encontra a oscilação de convecção a +313 s; nos primeiros 150 s, T_p não passa de 60 °C.

Tabela 7. Seção 9 do script (ensaio completo; M_p e t_p no 1º degrau; *pico de ruído a +313 s).

	Com AW	Sem AW
ISE (2100–3000 s) [°C²·s]	2997,8	2937,6
ISE calculado no firmware	2997,7	2937,6
Tempo saturado em U_{max}	92 s	216 s
M_p / t_p (1º degrau)	2,9 % / 312,9 s*	14,3 % / 39,4 s

Perturbações. Nos últimos 200 s de cada patamar com cooler constante, o erro médio ficou entre −0,12 e +0,01 °C (RMS de 0,08 a 0,15 °C): o integrador rejeita a perturbação constante sem erro em regime. Para manter 60 °C, u passou de 14570 (sem cooler) para ≈19300 (com $w = 680$), abaixo de U_{max}. Com a perturbação triangular sobra uma oscilação de ±0,4 a 0,8 °C no período do cooler (RMS de 0,24 °C a 50 °C e 0,35 °C a 60 °C). Um PI tem um único integrador e não zera o erro contra perturbações em rampa. Esse é o limite que motiva o observador de perturbações do Prj2CDig.

5.2 Degrau sem saturação

[Degrau de 50 para 52 °C (sem saturação, $u_{\text{pico}} \approx 14300 < U_{max}$) para comparar M_p e t_p experimentais com o projeto (10 %, 25 s; simulado: 10,1 %, 23,7 s).]

6. ANÁLISE CRÍTICA

- Nos degraus de 10 °C a resposta é limitada pela potência do aquecedor; M_p e t_p do projeto linear devem ser comparados com o degrau sem saturação.
- A chegada lenta com anti-windup poderia ser melhorada com anti-windup por realimentação (*back-calculation*), que descarrega o integrador gradualmente em vez de congelá-lo.
- O fit de 86 % é limitado pela convecção, não medida; uma caixa fechada (sugerida no roteiro) a reduziria, mas optou-se por manter a influência do ambiente. A parte do modelo que define o controle foi consistente e o experimento confirmou a simulação.
- $U_{max} = 21000$ leva os resistores a 0,31 W nos transitórios (0,28 W no regime a 60 °C com cooler), acima do nominal; uma montagem definitiva deveria usar resistores de 1/2 W.

7. CONCLUSÃO

O PI digital projetado pelo LGR discreto ($K = 2040$, $z_0 = 0{,}942$, $T_s = T_d/2 = 2{,}629$ s) seguiu a referência em onda quadrada de 50–60 °C, inclusive sob a perturbação máxima do cooler, com ISE de 2998 °C²·s. O anti-windup por integração condicional eliminou o sobrepasso causado pela saturação (de até 21,6 % para menos de 1 %), como previsto na simulação. A escolha de U_{max} dentro da janela (1), feita experimentalmente, foi decisiva: com menos potência a referência não é alcançada; com mais, o anti-windup não tem efeito. A perturbação constante foi rejeitada sem erro em regime, mas a triangular deixa erro residual, a motivação do Prj2CDig. [link do vídeo]

REFERÊNCIAS

Åström, K.J. and Hägglund, T. (2006). *Advanced PID Control*. ISA, Research Triangle Park, NC.

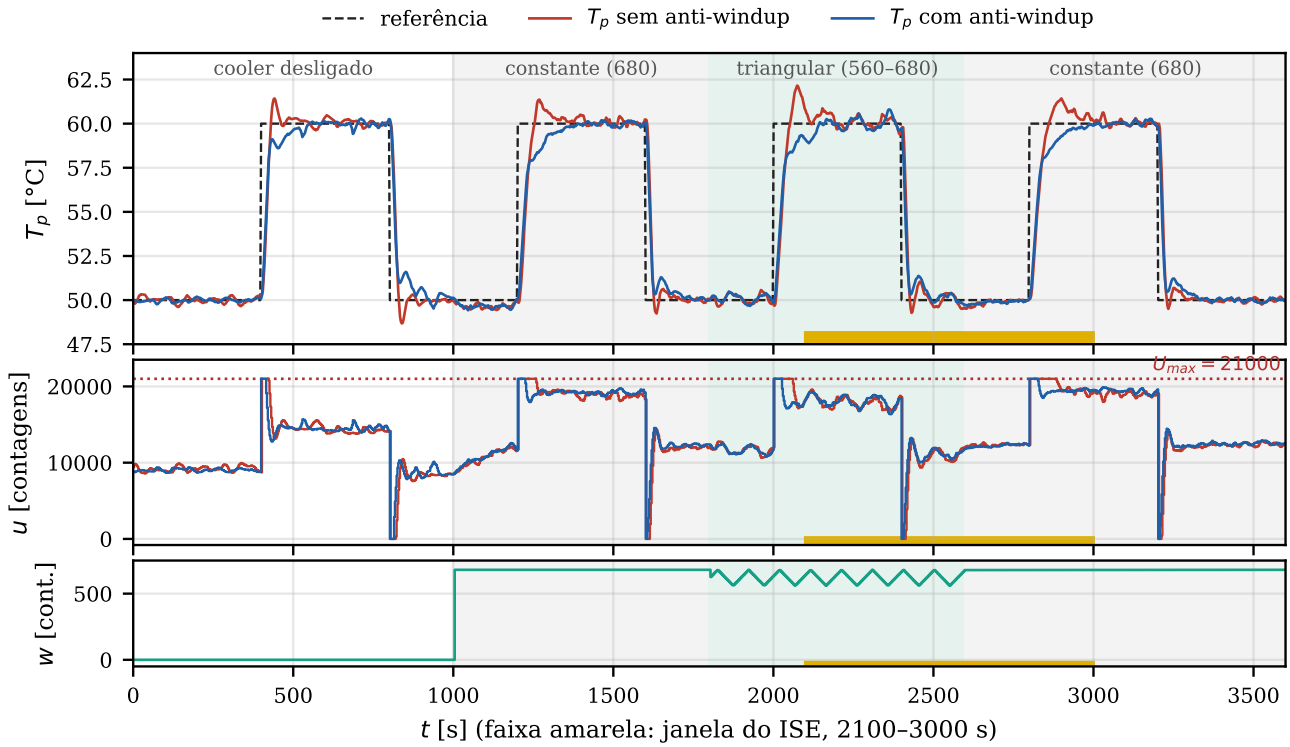


Figura 7. Ensaio do roteiro com e sem anti-windup ($U_{max} = 21000$, $K = 2040$, $z_0 = 0{,}942$, $T_s = 2{,}629\,\mathrm{s}$). Fundo cinza: cooler constante; verde: triangular.

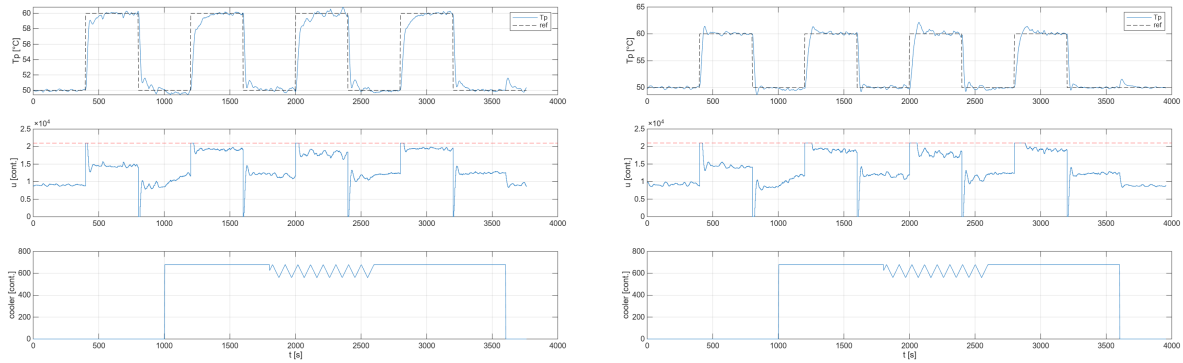


Figura 8. Ensaios analisados pela seção 9 do script: com anti-windup (esquerda) e sem anti-windup (direita). De cima para baixo: T_p e referência, u (saturação em 21000, tracejado) e cooler.

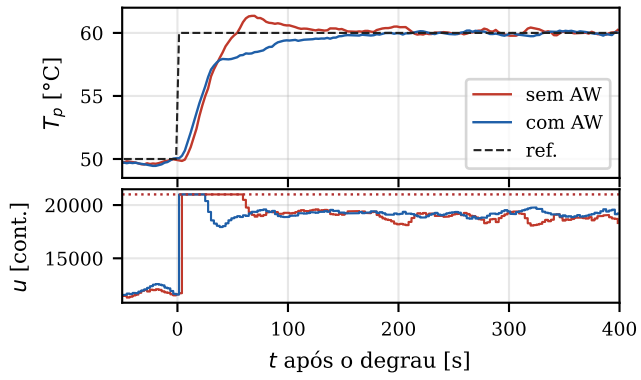


Figura 9. Detalhe do degrau 3 (cooler em 680): sem anti-windup, 55 s saturado e $T_p > 61\,^{\circ}\mathrm{C}$.

Bauchspiess, A. (2026). Prj1CDig – Controle PI Anti-WindUp de Processo Térmico (roteiro). URL `https://www.ene.unb.br/adolfo/Lectures/CDig.html`. ENE0167 Controle Digital, UnB.

Espressif Systems (2026). ESP-IDF Programming Guide v6.1: ADC Oneshot Mode Driver, ADC Calibration Driver e LED Control (LEDC). URL `https://docs.espressif.com/projects/esp-idf/`.

Ljung, L. (1999). *System Identification: Theory for the User*. Prentice Hall, Upper Saddle River, NJ, 2 edition.

Texas Instruments (2017). LM35 Precision Centigrade Temperature Sensors (SNIS159H). URL `https://www.ti.com/lit/ds/symlink/lm35.pdf`.

The MathWorks (2026). procest – Estimate process model using time or frequency data. URL `https://www.mathworks.com/help/ident/ref/procest.html`.

A.1 Firmware: configuração (`proj1cdig_config.h`)

```
/*
 * proj1cdig_config.h -- tudo o que muda de grupo para grupo ou depois de cada etapa.
 * Prj1CDig · ENE0167 Controle Digital · UnB
 *
 * Unidade do atuador: CONTAGENS do PWM de 16 bits do aquecedor (0..65535 = 0..12 V médio),
 * como pede o roteiro atualizado (passo 2: "PRBS implementado via PWM 16 bits").
 * Kp identificado (21/10/2026): parte rápida Kp·Tz/Tp1 ≈ 0,0017 °C/contagem; PI ≈ 2000 contagens/°C.
 * O cooler continua em 10 bits (0..1023): ele é perturbação, não precisa de resolução.
 */
#pragma once
#include "hal/adc_types.h"

/* ---------------- Pinos (ESP32-WROOM-32 DevKit, os "Dxx" da placa) ---------------- */
#define PIN_ADC_TP          35     /* D35 = ADC1_CH7: LM35 do processo (colado nos resistores) */
#define PIN_ADC_TA          34     /* D34 = ADC1_CH6: LM35 do ambiente */
#define PIN_PWM_HEATER      25     /* D25 → transistor do aquecedor   <<< CONFIRA ONDE ESTÁ LIGADO */
#define PIN_PWM_COOLER      26     /* D26 → transistor do cooler */
/* Não use para PWM: 34-39 (só entrada), 6-11 (flash), 0/2/5/12/15 (definem o boot), 1/3 (USB-serial). */

/* ---------------- ADC interno ---------------- */
#define ADC_ATENUACAO       ADC_ATTEN_DB_2_5  /* faixa útil ~100-1250 mV = 10-125 °C (0 dB iria só até ~95 °C) */
#define ADC_JANELA_MS       400    /* média de 400 ms por amostra: cancela 50 e 60 Hz */
#define ADC_LEITURAS_POR_MS 4      /* × 2 canais → ~1600 leituras de cada sensor por amostra */

/* Calibração de dois pontos (preencher na Fase 0, passo 0.6):
 *   T = CAL_GAIN * (V_mV / 10) + CAL_OFFSET   [°C]
 * Deixe 1.0 / 0.0 até calibrar. */
#define CAL_TP_GAIN         1.0f
#define CAL_TP_OFFSET       0.0f
#define CAL_TA_GAIN         1.0f
#define CAL_TA_OFFSET       0.0f

/* ---------------- Atuadores ---------------- */
#define HEATER_BITS         16     /* aquecedor: 16 bits exige f <= 80 MHz / 65536 = 1,22 kHz */
#define HEATER_FULL         65535
#define HEATER_FREQ_HZ      1000
#define PWM_BITS            10     /* cooler */
#define PWM_FULL            1023
#define COOLER_FREQ_HZ      25000

/* Limite do aquecedor. 3 × 150 Ω em paralelo a 12 V: 0,96 W em cada resistor a 100 %.
 * Ensaios de 21/10: U_MAX dentro da "janela" do anti-windup:
 *   - acima do u de regime no pior caso (60 °C com o cooler em 680: ~18000-19000; com 17000
 *     o ensaio parou em 55,9 °C), para a referência ser alcançável;
 *   - abaixo do u de pico que o PI pede no degrau de 10 °C (~30000-32000), para saturar nos
 *     transitórios e o efeito do anti-windup aparecer (com 34000 nunca saturava).
 * 21000 = 32 % de duty = 0,31 W por resistor (1,23 × a potência nominal de 1/4 W).
 * Este é o "Sat" REAL da bancada: use o MESMO valor em Sat no MATLAB/Simulink. */
#define U_MAX               21000  /* 32,0 % de 65535 */
#define U_MIN               0

/* Segurança: desliga o aquecedor se Tp passar disto. */
#define T_TRIP_C            95.0f

/* ---------------- Grupo (tabela do roteiro, ■1) ---------------- */
/* Grupo 2: P.O. 50 °C, Mp 10 %, Tp 25 s, w0 0,065 rad/s (período 96,7 s). */
#define GROUP_PO_C          50.0f
#define GROUP_W0_RAD_S      0.065f

/* ---------------- Perturbação (cooler) ---------------- */
/* Escala 0..1023 (10 bits). Zona morta medida (Etapa E): o motor parte em ~530 (52 %).
 * A senoide/triangular é COOLER_OFFSET ± COOLER_AMP, nunca cruzando a zona morta. */
#define COOLER_OFFSET       620    /* medido em 20/10: w = 680 derruba Tp ~5 °C a 50 °C em malha aberta */
#define COOLER_AMP          60     /* triangular 560..680; perturbação constante do ensaio = 680 */

/* ---------------- Identificação (PRBS) ---------------- */
#define PRBS_DEFAULT_U0     10200  /* u0 medido na Etapa F (20/10): Tp ≈ 49,8 °C (15,6 % de duty) */
#define PRBS_DEFAULT_DU     2000   /* ±5 °C em regime (ganho local ~0,0025); coleta de 20-21/10: 45,9-56,1 °C */
#define PRBS_DEFAULT_BIT_S  60     /* duração de cada nível; LFSR de 9 bits → 2^9-1 = 511 bits = 8,5 h */
#define PRBS_DEFAULT_T_AQUEC_S 2000 /* tempo em u0 antes do PRBS, para sair do ambiente e
                                       estabilizar no P.O. (= transit = 2000 do script do professor) */

/* ---------------- Controlador PI discreto ---------------- */
/* C(z) = PI_K * (z - PI_Z0) / (z - 1), período CTRL_TS_MS. Projeto do proj1cdig_m.m (21/10/2026)
```

```
 * sobre o modelo identificado (Tp - Ta): Kp = 0,00320 °C/cont, Tz = 821,9 s,
 * Tp1 = 1553,9 s, Tp2 = 34,46 s, Td = 5,257 s (fit 86 %). LGR discreto (ZOH, atraso de 2 amostras)
 * + refino: Mp = 10,1 %, tp = 23,7 s em simulação. */
#define CTRL_TS_MS          2629   /* Ts = Td/2 */
#define PI_K                2039.9595f /* contagens/°C */
#define PI_Z0               0.94216f

/* ---------------- Ensaio do roteiro (passo 8, Figura 5) -- comando "e" ----------------
 * Referência em onda quadrada entre P.O. e P.O. + ENSAIO_AMPL_C, meio período ENSAIO_MEIO_PERIODO_S.
 * Cooler: desligado até T_CONST1, constante (máximo da senoide) até T_TRI, triangular até
 * T_CONST2, constante até T_FIM. ISE = soma de e^2·Ts entre ISE_T0 e ISE_T1 (roteiro: 2100-3000 s). */
#define ENSAIO_AMPL_C          10.0f
#define ENSAIO_MEIO_PERIODO_S  400
#define ENSAIO_T_CONST1_S      1000
#define ENSAIO_T_TRI_S         1800
#define ENSAIO_T_CONST2_S      2600
#define ENSAIO_T_FIM_S         3600
#define ISE_T0_S               2100
#define ISE_T1_S               3000
```

A.2 Firmware: programa principal (`proj1cdig.c`)

```
/*
 * proj1cdig.c -- firmware do Prj1CDig (Controle PI Anti-Windup de Processo Térmico)
 * ENE0167 Controle Digital · UnB · ESP32-WROOM-32 (ADC interno) + 2x LM35 + transistores
 *
 * Um único firmware para todas as fases. O modo é escolhido por comando no serial
 * (115200 8N1, uma linha por comando). Digite "h" para ver a lista.
 *
 * Saída: linhas CSV começando com "D," -- lidas pelo proj1cdig_logger.py.
 *   D,t_s,k,modo,u,w,tp,ta,ref,aw
 *     t_s  tempo desde o boot [s]         k    índice da amostra no modo atual
 *     modo IDLE|SENS|OPEN|PRBS|CTRL       u    aquecedor [contagens 0..65535, limite U_MAX]
 *     w    cooler [contagens]             tp,ta temperaturas [°C]
 *     ref  referência [°C] (só CTRL)      aw   anti-windup 1/0
 * Linhas que começam com "#" são mensagens.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "esp_adc/adc_oneshot.h"
#include "esp_adc/adc_cali.h"
#include "esp_adc/adc_cali_scheme.h"
#include "esp_random.h"
#include "driver/ledc.h"
#include "driver/uart.h"
#include "esp_timer.h"
#include "esp_rom_sys.h"
#include "proj1cdig_config.h"

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

/* ================================================================== */
/* Estado compartilhado (protegido por g_mtx)                         */
/* ================================================================== */
typedef enum { M_IDLE, M_SENS, M_OPEN, M_PRBS, M_CTRL } mode_t_;
static const char *MODE_NAME[] = { "IDLE", "SENS", "OPEN", "PRBS", "CTRL" };

typedef enum { W_OFF, W_CONST, W_SIN, W_TRI } wmode_t_;

static struct {
    mode_t_  mode;
    bool     restart;      /* comando novo: reinicia contadores do modo */
    int      u_open;       /* OPEN: duty fixo */
    int      prbs_u0, prbs_du, prbs_duracao_bit_s, prbs_tempo_aquecimento_s;
    float    ref;          /* CTRL */
    bool     aw;           /* anti-windup ligado */
    float    K, z0;        /* PI */
    wmode_t_ wmode;
    int      w_const;
    int64_t  w_t0_us;      /* início da perturbação periódica */
```

```
    bool     ensaio;       /* ensaio do roteiro (comando e) em andamento */
    float    e_lo, e_hi;   /* níveis da onda quadrada de referência */
    int      e_meio;       /* meio período da onda quadrada [s] */
} g = {
    .mode = M_IDLE, .restart = true,
    .prbs_u0 = PRBS_DEFAULT_U0, .prbs_du = PRBS_DEFAULT_DU,
    .prbs_duracao_bit_s = PRBS_DEFAULT_BIT_S, .prbs_tempo_aquecimento_s = PRBS_DEFAULT_T_AQUEC_S,
    .ref = GROUP_PO_C, .aw = true, .K = PI_K, .z0 = PI_Z0,
    .wmode = W_OFF, .e_meio = ENSAIO_MEIO_PERIODO_S,
};
static SemaphoreHandle_t g_mtx;
#define LOCK()   xSemaphoreTake(g_mtx, portMAX_DELAY)
#define UNLOCK() xSemaphoreGive(g_mtx)

static volatile int g_w_now = 0;   /* duty atual do cooler, para o log */

/* ================================================================== */
/* PWM                                                                */
/* ================================================================== */
static void pwm_init(void)
{
    ledc_timer_config_t th = {
        .speed_mode = LEDC_LOW_SPEED_MODE, .timer_num = LEDC_TIMER_0,
        .duty_resolution = LEDC_TIMER_16_BIT, .freq_hz = HEATER_FREQ_HZ,   /* aquecedor: 16 bits */
        .clk_cfg = LEDC_AUTO_CLK };
    ESP_ERROR_CHECK(ledc_timer_config(&th));
    ledc_timer_config_t tc = th;
    tc.timer_num = LEDC_TIMER_1; tc.freq_hz = COOLER_FREQ_HZ;
    tc.duty_resolution = LEDC_TIMER_10_BIT;                                  /* cooler: 10 bits a 25 kHz */
    ESP_ERROR_CHECK(ledc_timer_config(&tc));

    ledc_channel_config_t ch = {
        .speed_mode = LEDC_LOW_SPEED_MODE, .channel = LEDC_CHANNEL_0,
        .timer_sel = LEDC_TIMER_0, .intr_type = LEDC_INTR_DISABLE,
        .gpio_num = PIN_PWM_HEATER, .duty = 0, .hpoint = 0 };
    ESP_ERROR_CHECK(ledc_channel_config(&ch));
    ch.channel = LEDC_CHANNEL_1; ch.timer_sel = LEDC_TIMER_1; ch.gpio_num = PIN_PWM_COOLER;
    ESP_ERROR_CHECK(ledc_channel_config(&ch));
}

static int clampi(int v, int lo, int hi) { return v < lo ? lo : (v > hi ? hi : v); }

static void heater_set(int u)
{
    u = clampi(u, U_MIN, U_MAX);              /* limite físico, sempre */
    ledc_set_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_0, u);
    ledc_update_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_0);
}

static void cooler_set(int w)
{
    w = clampi(w, 0, PWM_FULL);
    ledc_set_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_1, w);
    ledc_update_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_1);
    g_w_now = w;
}

/* ================================================================== */
/* ADC interno da ESP32 (ADC1: continua funcionando com o WiFi ligado)      */
/* ================================================================== */
/* O "filtro" da bancada (75 Ω + 1 μF) é o damper da Fig. 13 do datasheet do LM35: protege o
 * sensor contra a capacitância do fio, mas quase não filtra (canto em ~2 kHz). A filtragem é
 * digital: cada amostra é a média de ADC_JANELA_MS ms de leituras. 400 ms contém um número
 * inteiro de ciclos de 50 e de 60 Hz, que se cancelam na média; e a fase de cada rajada é
 * sorteada para o PWM do aquecedor não ficar "congelado" sempre no mesmo ponto.            */
static adc_oneshot_unit_handle_t s_adc;
static adc_channel_t s_ch_tp, s_ch_ta;
static float s_mv_a = 1250.0f / 4095.0f, s_mv_b = 0.0f;   /* mV = a·raw + b */
static bool s_adc_ok = false;
static const float s_cal[4] = { CAL_TP_GAIN, CAL_TP_OFFSET, CAL_TA_GAIN, CAL_TA_OFFSET };
static esp_err_t adc_init(void)
{
    adc_unit_t u1, u2;
    esp_err_t e = adc_oneshot_io_to_channel(PIN_ADC_TP, &u1, &s_ch_tp);
    if (e == ESP_OK) e = adc_oneshot_io_to_channel(PIN_ADC_TA, &u2, &s_ch_ta);
    if (e != ESP_OK) return e;
    if (u1 != ADC_UNIT_1 || u2 != ADC_UNIT_1) return ESP_ERR_INVALID_ARG;   /* ADC2 conflita com WiFi */
    adc_oneshot_unit_init_cfg_t uc = { .unit_id = ADC_UNIT_1, .ulp_mode = ADC_ULP_MODE_DISABLE };
    if ((e = adc_oneshot_new_unit(&uc, &s_adc)) != ESP_OK) return e;
```

```
135     adc_oneshot_chan_cfg_t cc = { .atten = ADC_ATENUACAO, .bitwidth = ADC_BITWIDTH_12 };
136     if ((e = adc_oneshot_config_channel(s_adc, s_ch_tp, &cc)) != ESP_OK) return e;
137     if ((e = adc_oneshot_config_channel(s_adc, s_ch_ta, &cc)) != ESP_OK) return e;
138
139     /* Calibração de fábrica (eFuse), que no ESP32 é uma reta: extraímos a e b para aplicar
140      * na MÉDIA em ponto flutuante, sem perder a resolução que a média ganhou. */
141     bool fabrica = false;
142 #if ADC_CALI_SCHEME_LINE_FITTING_SUPPORTED
143     adc_cali_line_fitting_config_t lc = {
144         .unit_id = ADC_UNIT_1, .atten = ADC_ATENUACAO, .bitwidth = ADC_BITWIDTH_12,
145 #if CONFIG_IDF_TARGET_ESP32
146         .default_vref = 1100,
147 #endif
148     };
149     adc_cali_handle_t cal = NULL;
150     if (adc_cali_create_scheme_line_fitting(&lc, &cal) == ESP_OK) {
151         int v0 = 0, v1 = 0;
152         adc_cali_raw_to_voltage(cal, 500, &v0);
153         adc_cali_raw_to_voltage(cal, 3500, &v1);
154         s_mv_a = (v1 - v0) / 3000.0f;
155         s_mv_b = v0 - s_mv_a * 500.0f;
156         adc_cali_delete_scheme_line_fitting(cal);
157         fabrica = true;
158     }
159 #endif
160     if (fabrica) printf("#ADC calibracao de fabrica: mV = %.5f*raw %+.2f\n", s_mv_a, s_mv_b);
161     else         printf("#ADC sem calibracao de fabrica: reta nominal (a calibracao de 2 pontos corrige)\n");
162     s_adc_ok = true;
163     return ESP_OK;
164 }
165
166 static void read_temps(float *tp, float *ta)
167 {
168     int64_t sp = 0, sa = 0;
169     int np = 0, na = 0;
170     if (s_adc_ok) {
171         TickType_t t = xTaskGetTickCount();
172         for (int i = 0; i < ADC_JANELA_MS; i++) {
173             esp_rom_delay_us(esp_random() % 850);          /* fase sorteada ao longo do período de 1 ms do PWM de 1 kHz */
174             for (int j = 0; j < ADC_LEITURAS_POR_MS; j++) {
175                 int r;
176                 if (adc_oneshot_read(s_adc, s_ch_tp, &r) == ESP_OK) { sp += r; np++; }
177                 if (adc_oneshot_read(s_adc, s_ch_ta, &r) == ESP_OK) { sa += r; na++; }
178             }
179             xTaskDelayUntil(&t, pdMS_TO_TICKS(1));
180         }
181     }
182     float vp = np ? s_mv_a * ((float)sp / np) + s_mv_b : NAN;
183     float va = na ? s_mv_a * ((float)sa / na) + s_mv_b : NAN;
184     *tp = s_cal[0] * (vp / 10.0f) + s_cal[1];
185     *ta = s_cal[2] * (va / 10.0f) + s_cal[3];
186 }
187
188 /* ================================================================== */
189 /* PRBS -- LFSR de 9 bits, x^9 + x^5 + 1, sequência-m de 511 bits     */
190 /* ================================================================== */
191 static uint16_t s_lfsr = 0x1A5;
192 static int prbs_next_bit(void)
193 {
194     int b = ((s_lfsr >> 0) ^ (s_lfsr >> 4)) & 1;
195     s_lfsr = (s_lfsr >> 1) | (b << 8);
196     return s_lfsr & 1;
197 }
198
199 /* ================================================================== */
200 /* PI discreto anti-windup                                            */
201 /*   C(z) = K (z - z0)/(z - 1) = K + K(1 - z0)/(z - 1)                */
202 /*   u(k)   = K e(k) + I(k)                                           */
203 /*   I(k+1) = I(k) + K(1 - z0) e(k)     (congelado se saturado e AW ligado)  */
204 /* ================================================================== */
205 typedef struct { float I; } pi_t;
206
207 static int pi_step(pi_t *pi, float e, float K, float z0, bool aw, bool *sat)
208 {
209     float u = K * e + pi->I;
210     bool hi = u > U_MAX, lo = u < U_MIN;
211     *sat = hi || lo;
212     /* integração condicional: só para de integrar se o erro empurra mais para a saturação */
213     bool freeze = aw && ((hi && e > 0) || (lo && e < 0));
```

```
214     if (!freeze) pi->I += K * (1.0f - z0) * e;
215     if (hi) u = U_MAX;
216     if (lo) u = U_MIN;
217     return (int)lroundf(u);
218 }
219 
220 #define W_PERT_MAX (COOLER_OFFSET + COOLER_AMP)   /* perturbação constante do ensaio = pico da triangular */
221 
222 /* ================================================================== */
223 /* Tarefa do cooler: atualiza a perturbação a cada 200 ms, independente de Ts */
224 /* ================================================================== */
225 static void cooler_task(void *arg)
226 {
227     for (;;) {
228         LOCK();
229         wmode_t_ m = g.wmode; int wc = g.w_const; int64_t t0 = g.w_t0_us;
230         UNLOCK();
231         double t = (esp_timer_get_time() - t0) / 1e6;
232         double th = GROUP_W0_RAD_S * t;
233         int w = 0;
234         switch (m) {
235         case W_OFF:   w = 0; break;
236         case W_CONST: w = wc; break;
237         case W_SIN:   w = COOLER_OFFSET + (int)lround(COOLER_AMP * sin(th)); break;
238         case W_TRI:   w = COOLER_OFFSET + (int)lround(COOLER_AMP * (2.0 / M_PI) * asin(sin(th))); break;
239         }
240         cooler_set(w);
241         vTaskDelay(pdMS_TO_TICKS(200));
242     }
243 }
244 
245 /* ================================================================== */
246 /* Tarefa principal: amostragem, modos, log                            */
247 /* ================================================================== */
248 static void control_task(void *arg)
249 {
250     pi_t pi = { 0 };
251     int u = 0, k = 0, prbs_u = 0, prbs_bit = 0;
252     int64_t t_ens0 = 0; int fase = -1; float ise = 0; bool ise_ok = false;   /* ensaio do roteiro */
253     TickType_t last = xTaskGetTickCount();
254 
255     for (;;) {
256         LOCK();
257         mode_t_ m = g.mode;
258         bool restart = g.restart; g.restart = false;
259         int u_open = g.u_open;
260         int u0 = g.prbs_u0, du = g.prbs_du, duracao_bit_s = g.prbs_duracao_bit_s, tempo_aquecimento_s = g.prbs_tempo_aquecimento_s;
261         float ref = g.ref, K = g.K, z0 = g.z0; bool aw = g.aw;
262         bool ensaio = g.ensaio; float e_lo = g.e_lo, e_hi = g.e_hi; int e_meio = g.e_meio;
263         UNLOCK();
264 
265         int period_ms = (m == M_CTRL) ? CTRL_TS_MS : 1000;   /* Ti = 1 s; Ts no controle */
266 
267         if (restart) {
268             k = 0;
269             last = xTaskGetTickCount();
270             if (m == M_CTRL) pi.I = (float)u;   /* partida sem solavanco: começa no u atual */
271             if (m == M_PRBS) { prbs_bit = 1; prbs_u = u0; }
272             if (m == M_CTRL && ensaio) { t_ens0 = esp_timer_get_time(); fase = -1; ise = 0; ise_ok = false; }
273         }
274 
275         float tp, ta;
276         read_temps(&tp, &ta);
277 
278         switch (m) {
279         case M_IDLE:
280         case M_SENS:
281             u = 0; break;
282         case M_OPEN:
283             u = u_open; break;
284         case M_PRBS: {
285             int t = k;                          /* k em segundos (período 1 s) */
286             if (t < tempo_aquecimento_s) prbs_u = u0;
287             else if ((t - tempo_aquecimento_s) % duracao_bit_s == 0) {
288                 prbs_bit = prbs_next_bit();
289                 prbs_u = prbs_bit ? u0 + du : u0 - du;
290             }
291             if (t >= tempo_aquecimento_s + 511 * duracao_bit_s) {     /* sequência completa: volta ao P.O. */
292                 prbs_u = u0;
```

```
293                 if (t == tempo_aquecimento_s + 511 * duracao_bit_s) printf("#PRBS concluido em k=%d\n", k);
294             }
295             u = prbs_u;
296             break; }
297         case M_CTRL: {
298             bool sat;
299             double te = -1;
300             if (ensaio) {                       /* ensaio do roteiro: referência e cooler pelo relógio */
301                 te = (esp_timer_get_time() - t_ens0) / 1e6;
302                 if (te >= ENSAIO_T_FIM_S) {
303                     ref = e_lo;
304                     LOCK(); g.ensaio = false; g.ref = e_lo; g.wmode = W_OFF; UNLOCK();
305                     printf("#ENSAIO concluido: ISE(%d-%d s) = %.1f\n", ISE_T0_S, ISE_T1_S, ise);
306                     ensaio = false;
307                 } else {
308                     ref = (((int)(te / e_meio)) % 2 == 0) ? e_lo : e_hi;
309                     int f = te < ENSAIO_T_CONST1_S ? 0 : te < ENSAIO_T_TRI_S ? 1 : te < ENSAIO_T_CONST2_S ? 2 : 3;
310                     LOCK();
311                     g.ref = ref;
312                     if (f != fase) {
313                         g.w_t0_us = esp_timer_get_time();
314                         if (f == 0) g.wmode = W_OFF;
315                         else if (f == 2) g.wmode = W_TRI;
316                         else { g.wmode = W_CONST; g.w_const = W_PERT_MAX; }
317                     }
318                     UNLOCK();
319                     if (f != fase) {
320                         static const char *NOME[] = { "cooler desligado", "perturbacao constante",
321                                                       "perturbacao triangular", "perturbacao constante" };
322                         printf("#ENSAIO t=%.0f s: %s\n", te, NOME[f]);
323                         fase = f;
324                     }
325                 }
326             }
327             if (isnan(tp)) break;               /* sem leitura: mantém u */
328             float e = ref - tp;
329             if (ensaio && te >= ISE_T0_S && te < ISE_T1_S) ise += e * e * (period_ms / 1000.0f);
330             if (ensaio && te >= ISE_T1_S && !ise_ok) {
331                 ise_ok = true;
332                 printf("#ISE(%d-%d s) = %.1f\n", ISE_T0_S, ISE_T1_S, ise);
333             }
334             u = pi_step(&pi, e, K, z0, aw, &sat);
335             break; }
336         }
337 
338         /* proteção térmica */
339         if (!isnan(tp) && tp > T_TRIP_C && u > 0) {
340             u = 0;
341             LOCK(); g.mode = M_IDLE; g.restart = true; UNLOCK();
342             printf("#TRIP Tp=%.2f > %.1f: aquecedor desligado, modo IDLE\n", tp, T_TRIP_C);
343         }
344         heater_set(u);
345 
346         if (m != M_IDLE)
347             printf("D,%.3f,%d,%s,%d,%d,%.3f,%.3f,%.2f,%d\n",
348                    esp_timer_get_time() / 1e6, k, MODE_NAME[m], clampi(u, U_MIN, U_MAX),
349                    g_w_now, tp, ta, (m == M_CTRL) ? ref : NAN, aw ? 1 : 0);
350         k++;
351         xTaskDelayUntil(&last, pdMS_TO_TICKS(period_ms));
352     }
353 }
354 
355 /* ========================================================================== */
356 /* Comandos                                                                   */
357 /* ========================================================================== */
358 static void help(void)
359 {
360     printf("#--- proj1cdig: comandos (uma linha cada) ---\n"
361            "# x                      para tudo (aquecedor e cooler em 0)\n"
362            "# s                      so le sensores, aquecedor em 0 (log 1 s)\n"
363            "# o <u>                  malha aberta, aquecedor fixo em u (0..%d)\n"
364            "# p [u0 du duracao_bit_s tempo_aquecimento_s]\n"
365            "#                        PRBS: fica tempo_aquecimento_s em u0, depois alterna\n"
366            "#                        u0+du / u0-du, cada nivel durando duracao_bit_s\n"
367            "# c [ref]                malha fechada PI, Ts=%d ms (ref padrao %.1f)\n"
368            "# r <ref>                degrau de referencia (em CTRL)\n"
369            "# a <0|1>                anti-windup desligado/ligado\n"
370            "# e [r_baixa r_alta meio_periodo_s]\n"
371            "#                        ensaio do roteiro (Fig. 5): onda quadrada (padrao P.O. / P.O.+10,\n"
```

```
372             "#                          400 s) + perturbacao constante e triangular, 3600 s, calcula o ISE\n"
373             "# k <K> <z0>               troca o PI sem regravar\n"
374             "# w0 | wc <w> | ws | wt    cooler: off | constante | senoide | triangular\n"
375             "# h                        esta ajuda\n",
376             U_MAX, CTRL_TS_MS, GROUP_PO_C);
377 }
378
379 static void print_state(void)
380 {
381     LOCK();
382     printf("#ESTADO modo=%s ref=%.2f K=%.4f z0=%.5f aw=%d prbs=[%d %d %d %d] wmode=%d ens=%d\n",
383            MODE_NAME[g.mode], g.ref, g.K, g.z0, g.aw, g.prbs_u0, g.prbs_du,
384            g.prbs_duracao_bit_s, g.prbs_tempo_aquecimento_s, g.wmode, g.ensaio);
385     UNLOCK();
386 }
387
388 static void handle(char *line)
389 {
390     char c = line[0], c2 = line[1];
391     char *args = line + 1;
392     LOCK();
393     if (c && strchr("xsopcrw", c)) g.ensaio = false;   /* qualquer outro modo/ajuste manual encerra o ensaio */
394     switch (c) {
395     case 'x': g.mode = M_IDLE; g.restart = true; g.wmode = W_OFF; break;
396     case 's': g.mode = M_SENS; g.restart = true; break;
397     case 'o': g.u_open = clampi(atoi(args), 0, U_MAX); g.mode = M_OPEN; g.restart = true; break;
398     case 'p': {
399         int a[4] = { g.prbs_u0, g.prbs_du, g.prbs_duracao_bit_s, g.prbs_tempo_aquecimento_s };
400         sscanf(args, "%d %d %d %d", &a[0], &a[1], &a[2], &a[3]);
401         if (a[0] + a[1] > U_MAX || a[0] - a[1] < 0 || a[2] < 1 || a[3] < 0) {
402             printf("#ERRO PRBS fora da faixa 0..%d\n", U_MAX); break;
403         }
404         g.prbs_u0 = a[0]; g.prbs_du = a[1]; g.prbs_duracao_bit_s = a[2]; g.prbs_tempo_aquecimento_s = a[3];
405         g.mode = M_PRBS; g.restart = true;
406         printf("#PRBS u0=%d du=%d duracao_bit=%d s tempo_aquecimento=%d s, duracao total %.2f h\n",
407                a[0], a[1], a[2], a[3], (a[3] + 511.0 * a[2]) / 3600.0);
408         break; }
409     case 'c': { float r; if (sscanf(args, "%f", &r) == 1) g.ref = r;
410                 g.mode = M_CTRL; g.restart = true; break; }
411     case 'r': { float r; if (sscanf(args, "%f", &r) == 1) g.ref = r; break; }
412     case 'a': g.aw = atoi(args) != 0; break;
413     case 'e': {
414         float a[3] = { GROUP_PO_C, GROUP_PO_C + ENSAIO_AMPL_C, ENSAIO_MEIO_PERIODO_S };
415         sscanf(args, "%f %f %f", &a[0], &a[1], &a[2]);
416         if (a[2] < 10) { printf("#ERRO meio periodo menor que 10 s\n"); break; }
417         g.e_lo = a[0]; g.e_hi = a[1]; g.e_meio = (int)a[2]; g.ref = a[0];
418         g.ensaio = true; g.wmode = W_OFF; g.mode = M_CTRL; g.restart = true;
419         printf("#ENSAIO iniciado: ref %.1f/%.1f C, meio periodo %d s, %d s no total, ISE entre %d e %d s\n",
420                a[0], a[1], (int)a[2], ENSAIO_T_FIM_S, ISE_T0_S, ISE_T1_S);
421         break; }
422     case 'k': { float K, z; if (sscanf(args, "%f %f", &K, &z) == 2) { g.K = K; g.z0 = z; } break; }
423     case 'w':
424         g.w_t0_us = esp_timer_get_time();
425         if (c2 == '0') g.wmode = W_OFF;
426         else if (c2 == 'c') { g.w_const = clampi(atoi(line + 2), 0, PWM_FULL); g.wmode = W_CONST; }
427         else if (c2 == 's') g.wmode = W_SIN;
428         else if (c2 == 't') g.wmode = W_TRI;
429         break;
430     case 'h': case '?': UNLOCK(); help(); return;
431     default: UNLOCK(); printf("#? comando desconhecido: %s (h = ajuda)\n", line); return;
432     }
433     UNLOCK();
434     print_state();
435 }
436
437 static void uart_task(void *arg)
438 {
439     char line[64]; int n = 0; uint8_t ch;
440     for (;;) {
441         if (uart_read_bytes(UART_NUM_0, &ch, 1, pdMS_TO_TICKS(100)) == 1) {
442             if (ch == '\r' || ch == '\n') {
443                 if (n) { line[n] = 0; handle(line); n = 0; }
444             } else if (n < (int)sizeof(line) - 1) line[n++] = (char)ch;
445         }
446     }
447 }
448
449 void app_main(void)
450 {
```

```
    g_mtx = xSemaphoreCreateMutex();
    pwm_init();
    heater_set(0); cooler_set(0);
    ESP_ERROR_CHECK(uart_driver_install(UART_NUM_0, 256, 0, 0, NULL, 0));

    printf("#proj1cdig iniciado. U_MAX=%d Ts=%d ms PO=%.1f w0=%.3f\n",
           U_MAX, CTRL_TS_MS, GROUP_PO_C, GROUP_W0_RAD_S);
    esp_err_t e = adc_init();
    if (e != ESP_OK)
        printf("#ERRO ADC: os GPIO %d e %d precisam ser do ADC1 (32 a 39) (%s)\n",
               PIN_ADC_TP, PIN_ADC_TA, esp_err_to_name(e));
    else
        printf("#ADC ok: Tp no GPIO%d, Ta no GPIO%d\n", PIN_ADC_TP, PIN_ADC_TA);
    printf("#H,t_s,k,modo,u,w,tp,ta,ref,aw\n");
    help();

    xTaskCreate(control_task, "ctrl", 4096, NULL, 5, NULL);
    xTaskCreate(cooler_task, "cooler", 3072, NULL, 4, NULL);
    xTaskCreate(uart_task, "uart", 3072, NULL, 3, NULL);
}
```

A.3 MATLAB: identificação e projeto (`proj1cdig_m.m`)

```
% proj1cdig_m.m -- Prj1CDig: identificação + projeto do PI anti-windup discreto
% ENE0167 Controle Digital · UnB
%
% Baseado no Prj1CDig_m.m do Prof. Adolfo (Referencias/Prj1CDig), adaptado para:
%   * ler o CSV do proj1cdig_logger.py em vez do ma31500.mat;
%   * calcular o ponto de partida do PI pelo LGR (condição de ângulo e de módulo),
%     contínuo e discreto, antes de abrir o sisotool;
%   * simular o PI discreto com saturação COM e SEM anti-windup (mesma lei do firmware);
%   * imprimir as linhas prontas para colar em firmware/main/proj1cdig_config.h;
%   * medir Mp e tp no ensaio real em malha fechada.
%
% Rode seção por seção (Ctrl+Enter). Copie Prj1CDig.slx e Termico_FP.slx da pasta
% Referencias para esta pasta (não edite os originais).
%
% Unidade do atuador: contagens do PWM de 16 bits, 0..65535 (= 0..12 V médio), como pede o
% roteiro atualizado. Kp sai em °C/contagem (~0,003) e o ganho do PI em contagens/°C (~2000).

clear; close all; clc

%% 0. Parâmetros do grupo  <<< TROQUE AQUI >>>
GRUPO = 2;                    % grupo 2: PO 50 °C, Mp 10 %, Tp 25 s, w0 0,065
ARQ_PRBS = 'dados/proj1cdig_prbs_2026-10-20_23-26-20.csv';
ARQ_MF   = '';                % CSV do ensaio em malha fechada (seção 9), '' para pular
Sat      = 21000;             % = U_MAX do firmware (3 × 150 Ω a 12 V: 32 % de 65535, 0,31 W por resistor)
ABRIR_SISOTOOL = false;       % true para abrir o Control System Designer nas seções 5 e 6
U0_MEDIDO = 10200;            % u que deixou Tp estável no P.O. (Etapa F, 20/10)

g      = mod(GRUPO-1, 10) + 1;
tabW0  = [0.065 0.065 0.065 0.065 0.065 0.098 0.098 0.098 0.098 0.098];
tabPO  = [45 50 55 60 45 50 55 60 45 50];
tabMp  = [10 10 10 25 25 30 30 30 15 15];
tabTp  = [25 25 25 30 25 25 30 30 25 30];
w0 = tabW0(g); PO = tabPO(g); Mp_esp = tabMp(g); Tp_esp = tabTp(g);
fprintf('Grupo %d: PO = %g °C, Mp = %g %%, Tp = %g s, w0 = %g rad/s\n', GRUPO, PO, Mp_esp, Tp_esp, w0);

%% 1. Carregar a coleta PRBS
Ti = 1;                                     % período de amostragem da identificação
T  = readtable(ARQ_PRBS, 'TextType', 'string');
T  = T(T.modo == "PRBS", :);
um = T.u;  ym = T.tp;  tam = T.ta;
tm = (0:numel(um)-1)' * Ti;
Ta = mean(tam, 'omitnan');                  % temperatura ambiente média do ensaio
fprintf('%d amostras (%.2f h). Ta média = %.2f °C (min %.2f, max %.2f)\n', ...
        numel(um), numel(um)/3600, Ta, min(tam), max(tam));

figure('Name','Coleta PRBS');
subplot(311); plot(tm/3600, ym); ylabel('Tp [°C]'); grid on; title('Coleta PRBS')
subplot(312); plot(tm/3600, tam); ylabel('Ta [°C]'); grid on
subplot(313); plot(tm/3600, um); ylabel('u [cont.]'); xlabel('t [h]'); grid on

%% 2. Identificação P2DZ (procest)
transit = 2000;                             % descarta o aquecimento até o P.O. (= warm_s)
USAR_TP_MENOS_TA = true;                    % identifica Tp - Ta: tira a deriva da sala
                                            % (ex.: queda de ~0,9 °C em Ta ao amanhecer, na coleta de 20-21/10)
```

```
ui = um(transit:end);
if USAR_TP_MENOS_TA, yi = ym(transit:end) - tam(transit:end); else, yi = ym(transit:end); end
uid = ui - mean(ui);  yid = yi - mean(yi);

% Chute inicial e limites na faixa do roteiro. Sem eles o procest cai num mínimo local
% (Tz = 0, Tp1 ~ 40 s, Td ~ 2 s, fit menor) que perde a cauda lenta da resposta.
ini = idproc('P2DZ');
ini.Structure.Kp.Value  = 0.002;  ini.Structure.Kp.Minimum  = 0;
ini.Structure.Tp1.Value = 300;    ini.Structure.Tp1.Minimum = 50;  ini.Structure.Tp1.Maximum = 2000;
ini.Structure.Tp2.Value = 40;     ini.Structure.Tp2.Minimum = 5;   ini.Structure.Tp2.Maximum = 200;
ini.Structure.Tz.Value  = 200;    ini.Structure.Tz.Minimum  = 0;   ini.Structure.Tz.Maximum  = 2000;
ini.Structure.Td.Value  = 4;      ini.Structure.Td.Minimum  = 0;   ini.Structure.Td.Maximum  = 15;

melhor = struct('fit', -Inf);
for i = 1:19                                  % partições identificação/validação
    len_id = ceil((100 - 5*i)/100 * numel(uid));
    data  = iddata(yid(1:len_id),   uid(1:len_id),   Ti);
    datav = iddata(yid(len_id:end), uid(len_id:end), Ti);
    if numel(datav.y) < 600, continue, end       % validação curta demais
    ge = procest(data, ini);
    [~, fit] = compare(datav, ge);
    fprintf('  partição %2d%% validação: fit = %6.2f %%  Td = %5.2f s\n', 5*i, fit, ge.Td);
    if fit > melhor.fit, melhor = struct('fit', fit, 'ge', ge, 'i', i, 'datav', datav); end
end
ge = melhor.ge;
figure('Name','Validação'); compare(melhor.datav, ge);
title(sprintf('Melhor partição: %d%% validação, fit %.2f %%', 5*melhor.i, melhor.fit))

Kp = ge.Kp; Tz = ge.Tz; Tp1 = ge.Tp1; Tp2 = ge.Tp2; Td = ge.Td;
if Tp2 > Tp1, [Tp1, Tp2] = deal(Tp2, Tp1); end   % Tp1 = polo lento (ambiente)
fprintf('\nModelo: Kp=%.5f °C/cont  Tz=%.2f s  Tp1=%.2f s  Tp2=%.2f s  Td=%.3f s\n', Kp, Tz, Tp1, Tp2, Td);
if Td < 3 || Td > 9
    warning('Td = %.2f s fora da faixa esperada pelo roteiro (4-8 s). Confira ruído e fixação.', Td);
end

G = tf(Kp*[Tz 1], conv([Tp1 1], [Tp2 1]), 'InputDelay', Td);

%% 3. Variáveis usadas por Termico_FP.slx e Prj1CDig.slx
Ts = Ti;                      % Termico_FP usa "Ts" = 1 s (a seção 6 redefine Ts!)
W = 0;
Am = (Tz - Tp1)/(Tp2 - Tp1);  Bm = 1 - Am;      % forma modal
Ct = 0.0002; Ka = 1; Kw = 4; Kt = 1/(Ct*Tp1) - Ka - Kw;   % (Ka+Kt+Kw)*Ct = 1/Tp1
Kp2 = 1/(Kt*Ct*Tp1);                                      %#ok<NASGU>
if w0 < 0.08, AmpW0 = 0.02; else, AmpW0 = 0.02*0.981/0.654; end   % valores do roteiro
% Modelo caixa-cinza contra os dados medidos (figura útil no vídeo):
% open Termico_FP; sim Termico_FP
U0 = (P0 - Ta)/Kp;                            % u de regime no P.O. (estimativa)
fprintf('u estimado no P.O.: %.0f contagens (%.1f %% de duty). Sat = %d.\n', U0, U0/655.35, Sat);

%% 4. Especificações → polo dominante desejado
Mp = Mp_esp/100;
zeta = -log(Mp)/sqrt(pi^2 + log(Mp)^2);
wn   = pi/(Tp_esp*sqrt(1 - zeta^2));
sd   = -zeta*wn + 1i*wn*sqrt(1 - zeta^2);
fprintf('\nζ = %.4f  ωn = %.4f rad/s  polo desejado s = %.4f ± j%.4f\n', zeta, wn, real(sd), imag(sd));

%% 5. PI contínuo pelo LGR:  C(s) = KP (s + a)/s = KP + KI/s
Gs  = polyval(Kp*[Tz 1], sd) / polyval(conv([Tp1 1],[Tp2 1]), sd) * exp(-Td*sd);
phi = mod(-pi - angle(Gs) + angle(sd), 2*pi);   % ângulo que o zero do PI precisa dar
a   = -real(sd) + imag(sd)/tan(phi);
KP  = 1/abs((sd + a)/sd * Gs);
KI  = KP*a;
fprintf('PI contínuo (LGR): zero em s = %.5f, KP = %.3f cont/°C, KI = %.4f cont/(°C·s)\n', -a, KP, KI);
if a <= 0, warning('Zero do PI no semiplano direito: relaxe Tp ou Mp no sisotool.'); end
Cs = tf(KP*[1 a], [1 0]);
Tcl = feedback(Cs*G, 1);  S = stepinfo(Tcl);
fprintf('  → simulação linear: Mp = %.1f %%, tp = %.1f s (o zero e os outros polos mudam a resposta: ajuste no sisotool)\n', S.
    Overshoot, S.PeakTime);
if ABRIR_SISOTOOL, controlSystemDesigner('rlocus', G, Cs); end
% Depois de ajustar no sisotool, exporte C e atualize KP e KI antes de rodar o Prj1CDig.slx.
% Confira no diagrama se o bloco Ki está em paralelo (C = KP + KI/s), como assumido aqui.

%% 6. PI discreto pelo LGR em z:  C(z) = K (z - z0)/(z - 1),  Ts = Td/2
Ts  = round(Td/2, 3);                         % Ts = Td/2 (passo 6a do roteiro)
Gd  = G;  Gd.InputDelay = 2*Ts;               % atraso = exatamente 2 amostras: evita o zero
Gz  = absorbDelay(c2d(Gd, Ts, 'zoh'));        % espúrio (z ~ -300) do atraso fracionário
zd  = exp(sd*Ts);
Gzd = evalfr(Gz, zd);
phi = mod(-pi - angle(Gzd) + angle(zd - 1), 2*pi);
```

```
z0  = real(zd) - imag(zd)/tan(phi);
K   = 1/abs((zd - z0)/(zd - 1) * Gzd);
fprintf('\nTs = %.2f s. Polo desejado em z = %.4f ± j%.4f\n', Ts, real(zd), imag(zd));
fprintf('PI discreto (condição de ângulo): z0 = %.5f, K = %.3f\n', z0, K);
Cz = tf(K*[1 -z0], [1 -1], Ts);
S  = stepinfo(feedback(Cz*Gz, 1));
fprintf('  → Mp = %.1f %%, tp = %.1f s\n', S.Overshoot, S.PeakTime);

% Refino: varre z0 e K perto do ponto de partida, procurando Mp e tp da especificação.
best = [Inf z0 K];
for z0i = linspace(max(0.5, z0 - 0.15), 0.999, 80)
    for Ki = K*linspace(0.2, 2, 60)
        Tz_ = feedback(tf(Ki*[1 -z0i], [1 -1], Ts)*Gz, 1);
        if ~isstable(Tz_), continue, end
        Si = stepinfo(Tz_);
        c = ((Si.Overshoot - Mp_esp)/Mp_esp)^2 + ((Si.PeakTime - Tp_esp)/Tp_esp)^2;
        if c < best(1), best = [c z0i Ki]; end
    end
end
z0 = best(2); K = best(3);
Cz = tf(K*[1 -z0], [1 -1], Ts);
S  = stepinfo(feedback(Cz*Gz, 1));
fprintf('PI discreto refinado: z0 = %.5f, K = %.3f → Mp = %.1f %%, tp = %.1f s\n', z0, K, S.Overshoot, S.PeakTime);
figure('Name','LGR discreto'); rlocus(Cz*Gz); zgrid(zeta, wn*Ts); hold on
hd = plot(real(zd)*[1 1], [imag(zd) -imag(zd)], 'rx', 'MarkerSize', 12, 'LineWidth', 2, 'LineStyle', 'none');
title('LGR de C(z)G(z)')
pmf = pole(feedback(Cz*Gz, 1));                % polos de malha fechada com o K escolhido
hm = plot(real(pmf), imag(pmf), 'ms', 'MarkerSize', 10, 'LineWidth', 1.5, 'LineStyle', 'none');
legend([hd hm], {'polo desejado (Mp, tp)', 'polos de MF (K, z0 finais)'}, 'Location', 'northwest')
axis equal; xlim([-1.1 1.1]); ylim([-1.1 1.1])
fprintf('Polos de MF: %s\n', mat2str(pmf.', 4));
if ABRIR_SISOTOOL, controlSystemDesigner('rlocus', Gz, Cz); end
% Use o sisotool para justificar no vídeo a posição do zero e do ganho (passos 6b e 6c).

%% 7. Simulação discreta com saturação: com e sem anti-windup
% Modelo linear em torno do ponto de operação MEDIDO (u0 = U0_MEDIDO, Tp = P0):
%   Tp = P0 + y_desvio,  y_desvio = Gz * (u - U0_MEDIDO),  u saturado em [0, Sat]
% (o ganho local Kp é menor que (P0 - Ta)/u0, então (P0 - Ta)/Kp superestimaria u0)
U0 = U0_MEDIDO;
Ts = Gz.Ts;                                    % garante o Ts do PI discreto (a seção 3 faz Ts = 1)
[A,B,C,D] = ssdata(ss(Gz));
N  = round(900/Ts);  t = (0:N-1)'*Ts;
fprintf('\nu necessário em regime: %.0f para P0+5, %.0f para P0+10 (Sat = %d)\n', U0 + 5/Kp, U0 + 10/Kp, Sat);
for dr = [5 10]                                % degrau de 5 °C e o da Figura 5 (P0 → P0+10)
    figure('Name', sprintf('Anti-windup, degrau de %d °C', dr));
    for aw = [1 0]
        x = zeros(size(A,1), 1);  I = U0;  y = zeros(N,1); u = y;
        for k = 1:N
            y(k) = C*x + P0;
            r = P0 + dr*(t(k) >= 30);
            e = r - y(k);  uk = K*e + I;  hi = uk > Sat;  lo = uk < 0;
            if ~(aw && ((hi && e > 0) || (lo && e < 0))), I = I + K*(1 - z0)*e; end
            u(k) = min(max(uk, 0), Sat);
            x = A*x + B*(u(k) - U0);
        end
        subplot(211); hold on; stairs(t, y); subplot(212); hold on; stairs(t, u);
        fprintf('degrau %2d °C, AW=%d: pico %.2f °C (sobrepasso %5.1f %%), saturado %4.0f s, final %.2f °C\n', ...
                dr, aw, max(y), (max(y) - P0 - dr)/dr*100, sum(u >= Sat - 1)*Ts, y(end));
    end
    subplot(211); yline(P0 + dr, 'k--'); ylabel('Tp [°C]'); legend('com AW', 'sem AW', 'Location', 'southeast'); grid on
    title(sprintf('Degrau de %d °C a partir do P.O. (u0 = %d), saturação em %d', dr, U0, Sat))
    subplot(212); yline(Sat, 'r--'); ylabel('u [cont.]'); xlabel('t [s]'); grid on
end
% Para o passo 7 do roteiro, monte também a versão discreta no Simulink: copie
% Prj1CDig.slx, troque o PI contínuo por um Discrete Transfer Fcn / blocos com Ts,
% mantendo a saturação 0-Sat e o chaveamento do anti-windup
% (set_param('Prj1CDig/Anti-WindUp','sw','1') = sem AW, '0' = com AW).

%% 8. Linhas para colar em firmware/main/proj1cdig_config.h
fprintf('\n/* ---- gerado por proj1cdig_m.m em %s ---- */\n', datestr(now));
fprintf('#define GROUP_P0_C         %.1ff\n', P0);
fprintf('#define GROUP_W0_RAD_S     %.3ff\n', w0);
fprintf('#define CTRL_TS_MS         %d\n', round(Ts*1000));
fprintf('#define PI_K               %.4ff\n', K);
fprintf('#define PI_Z0              %.5ff\n', z0);
fprintf('/* ou, sem regravar: k %.4f %.5f */\n', K, z0);

%% 9. Ensaio em malha fechada: Mp, tp e ISE medidos
% Para o ensaio do roteiro (comando "e" do firmware, Figura 5): o arquivo começa no instante
```

```
% do comando, então t = 0 é o início do ensaio e o ISE é calculado entre 2100 e 3000 s.
if ~isempty(ARQ_MF)
    M = readtable(ARQ_MF, 'TextType', 'string');
    M = M(M.modo == "CTRL", :);
    i0 = find(M.k == 0, 1);  if isempty(i0), i0 = 1; end   % início do ensaio (k zera no comando)
    M = M(i0:end, :);
    t = M.t_s - M.t_s(1);
    dt = [diff(t); median(diff(t))];
    if t(end) >= 3000
        jise = t >= 2100 & t < 3000;
        ISE = sum((M.ref(jise) - M.tp(jise)).^2 .* dt(jise));
        fprintf('ISE (2100-3000 s) = %.1f °C²·s   (roteiro: típico 2000-4000 no Simulink)\n', ISE);
    end
    fprintf('Tempo com o aquecedor saturado: %.0f s\n', sum(dt(M.u >= Sat - 1)));
    kd = find(abs(diff(M.ref)) > 0.5, 1);        % primeiro degrau de referência
    if isempty(kd)
        warning('Nenhum degrau de referência no arquivo.');
    else
        r0 = M.ref(kd); r1 = M.ref(kd+1); t0 = t(kd+1);
        kprox = find(abs(diff(M.ref(kd+1:end))) > 0.5, 1);   % próximo degrau (onda quadrada)
        tfim = t0 + 600; if ~isempty(kprox), tfim = min(tfim, t(kd + kprox)); end
        jan = t >= t0 & t <= tfim;
        [yp, ip] = max(sign(r1 - r0) * M.tp(jan)); yp = sign(r1 - r0)*yp;
        tj = t(jan);
        fprintf('Ensaio: Mp = %.1f %% (esp. %g %%), tp = %.1f s (esp. %g s)\n', ...
            (yp - r1)/(r1 - r0)*100, Mp_esp, tj(ip) - t0, Tp_esp);
    end
    figure('Name','Malha fechada');
    subplot(311); plot(t, M.tp, t, M.ref, 'k--'); ylabel('Tp [°C]'); grid on; legend('Tp','ref')
    subplot(312); stairs(t, M.u); yline(Sat, 'r--'); ylabel('u [cont.]'); grid on
    subplot(313); plot(t, M.w); ylabel('cooler [cont.]'); xlabel('t [s]'); grid on
end
```